Numero di stati in un DFA nel caso peggiore

Vediamo ora quale è il **numero di stati in un DFA nel caso peggiore**.

Abbiamo il seguente lemma:

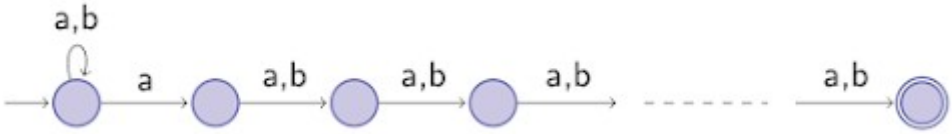
Per ogni $n \in N^+$, esiste un NFA con $n+1$ stati il cui DFA equivalente minimale ha almeno 2^n stati e una funzione di transizione totale.

In altre parole, esistono degli NFA i quali trasformati in DFA vedono crescere il numero di stati in maniera **esponenziale**.

Vediamo la *dimostrazione* per questo lemma.

Prendiamo in considerazione il linguaggio L: $L = L\left((a \mid b)^* a (a \mid b)^{n-1}\right)$, ovvero il linguaggio delle parole su $\{a, b\}$ che nella n-esima posizione a partire dal fondo possiede una a.

C'è un NFA che accetta il linguaggio L e ha esattamente $n+1$ stati:



Supponiamo, per assurdo, che esista un DFA minimale D che accetta L e ha $k < 2^n$ stati.

Ci sono esattamente 2^n parole distinte sull'insieme $\{a, b\}$ di lunghezza n.

Allora esistono due percorsi in D:

- La cui lunghezza è n
- Che rappresentano rispettivamente le parole w_1 e w_2 con $w_1 \neq w_2$
- Che **condividono almeno un nodo** (perché il DFA ha meno di 2^n stati)

Altrimenti, se non esistessero questi due percorsi, gli stati sarebbero come minimo 2^n.

Allora, per alcuni x_1, x_2, x, vale o:

- $w_1 = x_1 a x$ e $w_2 = x_2 b x$ oppure
- $w_1 = x_1 b x$ e $w_2 = x_2 a x$

Ovvero andiamo a pescare la prima posizione dal fondo per la quale le due parole sono differenti.

Senza perdita di generalità, supponiamo che $w_1 = x_1 a x$ e $w_2 = x_2 b x$.

Allora possiamo definire $w_1' = x_1 a b^{n-1} \in L(D)$.

Quindi lo stato raggiunto da w_1' in D è finale.

Ma questa è una contraddizione in quanto quello stato non può essere finale perché viene raggiunto anche da $x_2 b b^{n-1}$, che non appartiene al linguaggio $L(D)$.

Tutto il lemma gioca sul fatto che un DFA ha bisogno esattamente di tutti i possibili cammini che siano composti da $n-1$ alternanze di a e di b per esprimere lo stesso linguaggio espresso dall'NFA.

Dunque l'assurdità sta nel fatto che uno stato dovrebbe essere contemporaneamente finale e non finale, e questo perché abbiamo assunto che il DFA minimo abbia meno di 2^n nodi.

Pumping lemma per linguaggi regolari

Il "pumping lemma" per i linguaggi regolari è un teorema fondamentale nella teoria dei linguaggi formali. Esso fornisce una condizione che deve essere verificata da un linguaggio affinché possa essere considerato regolare.

Lemma: Sia L un linguaggio regolare. Allora:

- $\exists\, p \in N^+$ (detta "**pumping length**") tale che
- $\forall\, z \in L$ tale che $|z| > p$
- $\exists\ u, v, w$ tali che
  - $z = uvw$ e
  - $|uv| \leq p$ e
  - $|v| > 0$ e
  - $\forall\, i \in N,\ u v^i w \in L$

Dimostrazione:

- Sia L un linguaggio regolare.
- Allora esiste un DFA $D = (S, A, move_d, s_0, F)$ tale che $L = L(D)$.
- Sia $p = |S| - 1$.
- Allora tutti i percorsi da s_0 a uno stato finale che attraversano ogni stato al massimo una volta hanno lunghezza limitata da p (se la lunghezza della stringa è p, allora abbiamo visitato $p + 1 = |S|$ stati)
- Quindi, data $|z| > p$, per alcuni $a_1, \ldots, a_p$ e per qualche z', possiamo scomporre z in questa maniera $z = a_1 \cdots a_p z'$ e almeno uno stato, diciamo s^*, viene attraversato più di una volta lungo il cammino $a_1 \cdots a_p$.
- Allora esiste un ciclo in D che va da s^* a s^* e che rappresenta $a_{i+1} \cdots a_j$ per alcuni i e j tali che $i < j \leq p$.
- Allora prendendo come
  - $u = a_1 \cdots a_i$
  - $v = a_{i+1} \cdots a_j$
  - $w = \begin{cases} z' & \text{if } j = p \\ a_{j+1} \cdots a_p z' & \text{if } j < p \end{cases}$
- Allora:
  - $|uv| \leq p$ (perchè siamo sicuri di incontrare la seconda ricorrenza di s^* di entro p transizioni)
  - La lunghezza di v è almeno 1 (per definizione, un ciclo ha almeno una transizione, ed essendo un DFA allora sicuramente leggerà almeno un carattere)

- $u\,v^i\,w$ è accettata da D per ogni $i \in N$

A cosa serve questo lemma?
Se un linguaggio è regolare, allora vale *thesis*:

$$\exists p \in \mathbb{N}^+.\ \forall z \in \mathcal{L}:\ |z| > p.\ \exists u, v, w.\ P$$
where
$$P \equiv (z = uvw \text{ and } |uv| \leq p \text{ and } |v| > 0 \text{ and } \forall i \in \mathbb{N}. uv^i w \in \mathcal{L})$$

Dunque il lemma non può assolutamente essere usato per dimostrare che un certo linguaggio sia regolare.
È usato invece per verificare che un linguaggio non è regolare, infatti

- Assumiamo che L sia un linguaggio regolare
- Mostriamo che *thesis* è falso, ovvero che $not\,(thesis)$ è vero
- Per contraddizione possiamo concludere che L non è un linguaggio regolare

Dunque la $not\,(thesis)$ è:

$$\forall p \in \mathbb{N}^+.\ \exists z \in \mathcal{L}:\ |z| > p.\ \forall u, v, w.\ Q$$
where
$$Q \equiv (z = uvw \text{ and } |uv| \leq p \text{ and } |v| > 0) \text{ implies}$$
$$(\exists i \in \mathbb{N}. uv^i w \notin \mathcal{L})$$

Proviamo che il linguaggio $L = \{a^n b^n \mid n > 0\}$ non è regolare.
Intuitivamente, notiamo che lo stato in cui ci si trova dopo aver letto 2 occorrenze di a non può essere lo stesso stato in cui ci si trova dopo aver letto 3 occorrenze di a, questo perché non è possibile tener conto di quante a sono state lette, e dunque il numero di stati dovrebbe essere infinito in quanto n è infinito.
Vediamo come dimostrarlo tramite il pumping lemma.
Supponiamo che L sia un linguaggio regolare, e sia p un generico numero intero positivo.
Consideriamo $z = a^p b^p$, e quindi $|z| > p$.
Osserviamo che $\forall u, v, w \quad if\,(z = uvw \ \wedge\ |uv| \leq p \ \wedge\ |v| > 0)$:

- Allora v contiene solamente a, poichè $|uv| \leq p$
- E v contiene almeno una a, poichè $|v| > 0$

Quindi, per qualche $j > 0$, $u\,v^2\,w = a^p a^j b^p$
Allora $u\,v^2\,w \notin L$, il che contraddice il Pumping Lemma per i linguaggi regolari.

La proprietà più basilare di un DFA è che è finito: ha un numero finito di stati e nessun altro posto in cui memorizzare informazioni. Questo significa, ad esempio, che qualsiasi linguaggio che richieda un conteggio illimitato non può essere regolare. Un esempio di ciò è il linguaggio $\{a^n b^n \mid n > 0\}$, cioè, qualsiasi sequenza di a seguita da una sequenza dello stesso numero di b. Se dobbiamo decidere l'appartenenza a questo linguaggio con un DFA che legge l'input da sinistra a destra, dobbiamo, al momento in cui abbiamo letto tutte le a, sapere quant'è il loro numero, così da poterlo confrontare con il numero di b. Ma poiché un automa finito non può fare

conteggi arbitrariamente elevati, il linguaggio non è regolare. Un linguaggio simile che non è regolare è il linguaggio delle parentesi ben annidate. Tuttavia, se limitiamo la profondità di annidamento delle parentesi a una costante n, possiamo riconoscere questo linguaggio con un DFA che ha $n+1$ stati (da 0 a n), dove lo stato i corrisponde a i parentesi aperte non abbinate. Lo stato 0 è sia lo stato iniziale che l'unico stato accettante.

Chiusura rispetto all'unione dei linguaggi liberi

Vediamo ora la proprietà per la quale **la classe dei linguaggi liberi è chiusa rispetto all'operazione di unione insiemistica**, ovvero:

- Se L_1 e L_2 sono linguaggi liberi, allora $L_1 \cup L_2$ è un linguaggio libero

Dimostrazione:

- Siano L_1 e L_2 due linguaggi liberi
- Allora esistono due grammatiche libere $G_1=(V_1,T_1,S_1,P_1)$ e $G_2=(V_2,T_2,S_2,P_2)$ tali che $L_1=L(G_1)$ e $L_2=L(G_2)$
- Possiamo assumere che $(V_1\{T¿¿1) \cup (V_2\{T¿¿2)=\varnothing$, ovvero che G_1 e G_2 non condividono alcun non-terminale (visto che possiamo rinominarli a piacere)
- Sia $G=(V_1 \cup V_2 \cup \{S\}, T_1 \cup T_2, S, P_1 \cup P_2 \cup \{S \to S_1 | S_2\})$, con S nuovo simbolo non appartenente a $V_1 \cup V_2$
- Allora $L(G)$ è un linguaggio libero e $L(G)=L(G_1) \cup L(G_2)$

Verifichiamo che G è libera:

- Le produzioni di G sono quelle in $P_1 \cup P_2 \cup \{S \to S_1 | S_2\}$
- Le produzioni in P_1 e P_2 sono tutte nella forma $A \to w$ (perchè provengono da grammatiche libere)
- Le produzioni $S \to S_1$ e $S \to S_2$ hanno anche loro la forma $A \to w$
- Dunque G è una grammatica libera

Verifichiamo che vale $L(G)=L(G_1) \cup L(G_2)$:

- $w \in L(G)$
- Se e solo se esiste una derivazione $S \Rightarrow^* w$
- Se e solo se $S \Rightarrow S_1 \Rightarrow^* w$ oppure $S \Rightarrow S_2 \Rightarrow^* w$
- Se e solo se $S_1 \Rightarrow^* w$ oppure $S_2 \Rightarrow^* w$
- Se e solo se $w \in L(G_1)$ oppure $w \in L(G_2)$
- Se e solo se $w \in L(G_1) \cup L(G_2)$

Come mai insistiamo sul fatto che G_1 e G_2 non condividano non-terminali? Dobbiamo farlo per evitare di mescolare le produzioni di una grammatica con le produzioni dell'altra e ottenere derivazioni errate. Ecco un esempio:

- Take

$$\mathcal{G}_1: \begin{array}{lcl} S_1 & \to & aA \\ A & \to & a \end{array} \qquad \mathcal{G}_2: \begin{array}{lcl} S_2 & \to & bA \\ A & \to & b \end{array}$$

- Then $\mathcal{L}(\mathcal{G}_1) = \{aa\}$ and $\mathcal{L}(\mathcal{G}_2) = \{bb\}$.
- However, if we just put everything together and define

$$\mathcal{G}: \begin{array}{lcl} S & \to & S_1 \mid S_2 \\ S_1 & \to & aA \\ S_2 & \to & bA \\ A & \to & a \mid b \end{array}$$

- Then $\mathcal{L}(\mathcal{G}) = \{aa, ab, ba, bb\} \neq \mathcal{L}(\mathcal{G}_1) \cup \mathcal{L}(\mathcal{G}_2)$.
- Take

$$\mathcal{G}_1: \begin{array}{lcl} S_1 & \to & aA \\ A & \to & a \end{array} \qquad \mathcal{G}_2: \begin{array}{lcl} S_2 & \to & bA \\ A & \to & b \end{array}$$

- Then $\mathcal{L}(\mathcal{G}_1) = \{aa\}$ and $\mathcal{L}(\mathcal{G}_2) = \{bb\}$.
- The symbol A plays distinct roles in $\mathcal{G}_1$ and in $\mathcal{G}_2$
- To reflect this distinction, rather take

$$\mathcal{G}_1: \begin{array}{lcl} S_1 & \to & aA \\ A & \to & a \end{array} \qquad \mathcal{G}_2: \begin{array}{lcl} S_2 & \to & bA' \\ A' & \to & b \end{array}$$

- No mix up of productions now, and, by contruction, $\mathcal{L}(\mathcal{G}) = \{aa, bb\}$

Pumping lemma per linguaggi liberi

Ricordando il lemma della grammatica pulita:
Lemma: Sia L un linguaggio libero. Allora esiste una grammatica libera G tale che $L(G) = L \setminus \{\varepsilon\}$ e che:

- Non abbia ε-produzioni (ovvero non abbia produzioni della forma $A \to \varepsilon$)
- Non abbia produzioni unitarie (ovvero non abbia produzioni della forma $A \to B$)
- Non abbia non-terminali inutili (ovvero non abbia non-terminali che non compaiono mai in nessuna derivazione di alcune stringhe di terminali)

Possiamo facilmente notare che ogni produzione $A \to \beta$ in G è tale che o β è un singolo terminale oppure $|\beta| \geq 2$, in ogni caso β non è annullabile.

Abbiamo precedentemente visto il pumping lemma per i linguaggi regolari, vediamolo ora per i linguaggi liberi.

Il pumping lemma riguarda la presenza in ogni linguaggio context-free infinito (e in particolare in ogni linguaggio regolare infinito) di successioni di stringhe che presentano regolarità ben

definite. In particolare ogni stringa sufficientemente lunga di un tale linguaggio contiene una o più porzioni che possono essere ripetute un numero arbitrario di volte ottenendo ancora una stringa appartenente al linguaggio.

Lemma: Sia L un linguaggio libero. Allora:

- $\exists\, p \in N^+$ (detta "**pumping length**") tale che
- $\forall\, z \in L$ tale che $|z| > p$
- $\exists\ u, v, w, x, y$ tali che
  - $z = uvwxy$ e
  - $|vwx| \leq p$ e
  - $|vx| > 0$ e
  - $\forall\, i \in N,\ u v^i w x^i y \in L$

Dimostrazione:

- Sia L un linguaggio libero
- Il lemma tratta parole che sono più lunghe di p, con $p > 0$, e quindi che sono diverse da ε
- Possiamo quindi considerare una grammatica "pulita" G tale che $L(G) = L \text{¿} \{\varepsilon \text{¿}\}$
- Trasformiamo G in una grammatica in forma normale di Chomsky G', ovvero dove ogni produzione ha la forma
  - $A \to a$ oppure
  - $A \to A_1 A_2$

- Example
  - $S \to aSb \mid ab$
  - Both aSb and ab are to be changed. Pick up a new non-terminal for a, say A, and a new non-terminal for b, say B, and transform
  - $S \to ASB \mid AB, \quad A \to a, \quad B \to b$
  - ASB is longer than 2, pick up a new non-terminal for SB, say C, and transform
  - $S \to AC \mid AB, \quad C \to SB, \quad A \to a, \quad B \to b$

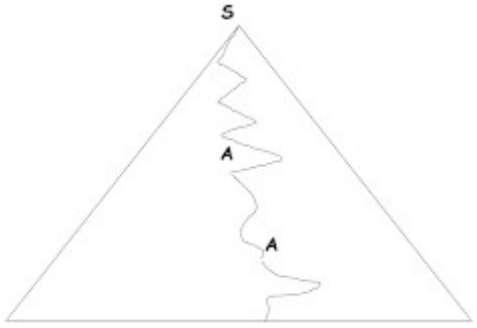
- Definiamo k come il numero di non-terminali in G'
- Osserviamo che ogni albero di derivazione per le parole in $L(G')$ ha:
  - Esattamente $2^0 = 1$ nodo al livello 0
  - Al massimo 2^1 nodi al livello 1
  - Al massimo 2^2 nodi al livello 2
  - …
  - Al massimo 2^j nodi al livello j
- Prendiamo $p = 2^{k+1}$
- Prendiamo $z \in L$ tale che $|z| > p$
- Allora l'albero di derivazione per z deve avere almeno $k+2$ livelli
- Allora ogni cammino più lungo dell'albero di derivazione per z ha un terminale all'ultimo livello e attraversa quindi almeno $k+1$ non-terminali

- Quindi c'è almeno un non-terminale che si verifica due o più volte lungo il percorso
- Consideriamo il percorso più lungo nell'albero e la coppia più profonda di occorrenze dello stesso non-terminale lungo quel percorso (ovvero scegli il non-terminale la cui seconda occorrenza viene trovata per prima andando dal basso verso l'alto)

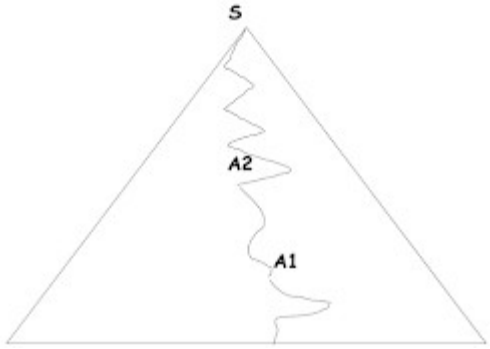
For instance, below the deepest pair is always the pair of As



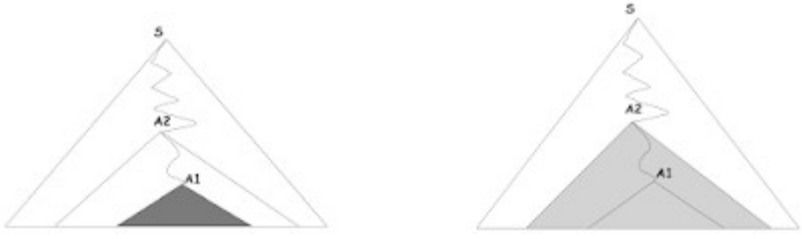
- Sia A il non-terminale della coppia più profonda di occorrenze dello stesso non-terminale lungo il percorso



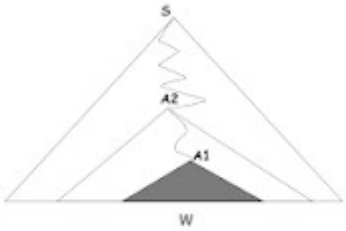
- Chiamiamo A_1 e A_2 le due occorrenze di A per distinguerle

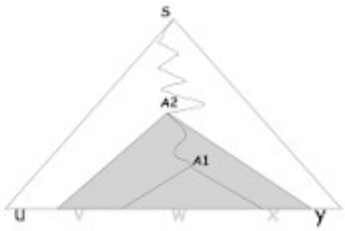


- Quindi ci i sono due sottoalberi distinti con radice in A: il “sottoalbero rosa” (più piccolo) e il “sottoalbero verde” (più grande)

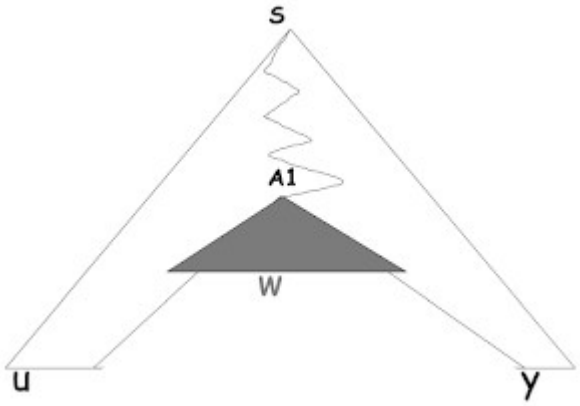


- Allora $z = uvwxy$

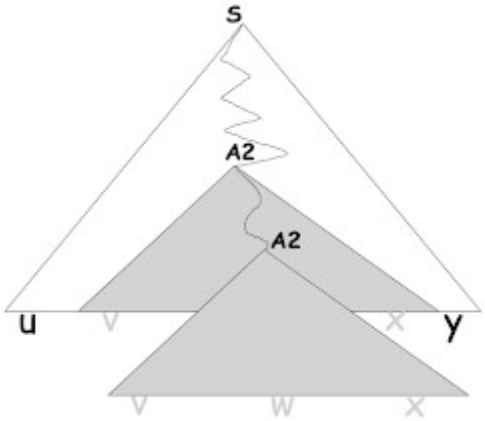




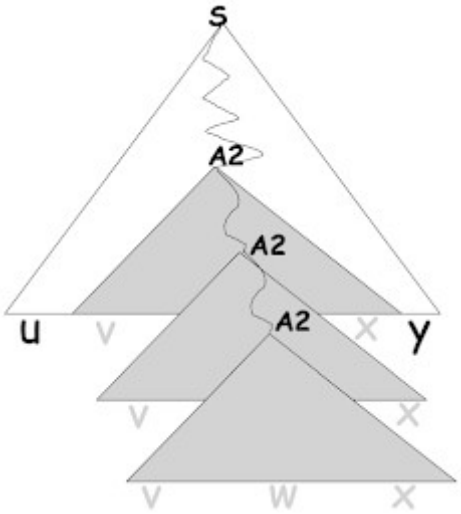
- E quindi $u\,v^0\,w\,x^0\,y \in L$



- E quindi $u\,v^2\,w\,x^2\,y \in L$



- E quindi $u\,v^3\,w\,x^3\,y \in L$



- Quindi $\forall\, i \in N,\, u\,v^i\,w\,x^i\,y \in L$
- $|vwx| \leq p$
    - Poiché abbiamo scelto la coppia più profonda di non-terminali ripetuti (A_1, A_2) lungo il percorso più lungo da S
    - Quindi lungo il percorso più lungo da A_2, sotto A_2, nessun non-terminale può verificarsi più di una volta
    - Quindi il sottoalbero radicato in A_2 ha al massimo $k+1$ livelli
    - Quindi la lunghezza della frontiera (il numero di foglie) del sottoalbero è limitato da $2^{k+1} = p$
- $|vx| > 0$
    - Poichè G' è una grammatica pulita
    - Dunque non può essere $A \Rightarrow^+ A$ ma piuttosto $A \Rightarrow^+ \alpha\, A\, \beta$ con almeno un terminale derivato da α o da β, dunque v e x non possono essere entrambe vuote

Questo ci conferma che nei linguaggi liberi si riescono a riconoscere dei bilanciamenti.

Per lo stesso discorso fatto per il pumping lemma nel caso dei linguaggi regolari, il pumping lemma per linguaggi liberi è utile per verificare che un linguaggio **non è libero**:

- Supporre che il linguaggio sia libero
- Dimostrare che la $not\,(PL\text{-}THESIS)$ è vera

La $PL\text{-}THESIS$ è la seguente:

- $\exists p \in \mathbb{N}^+.\ \forall z \in \mathcal{L}:\ |z| > p.\ \exists u, v, w, x, y.\ P$
- where
  - $P \equiv P1$ and $P2$ and $P3$ and $P4$
  - $P1 \equiv z = uvwxy$
  - $P2 \equiv |vwx| \leq p$
  - $P3 \equiv |vx| > 0$
  - $P4 \equiv \forall i \in \mathbb{N}. uv^i wx^i y \in \mathcal{L}$

La $not\,(PL\text{-}THESIS)$ invece è la seguente e proviamo a “semplifcarla”:

- not ($\exists p \in \mathbb{N}^+.\ \forall z \in \mathcal{L}:\ |z| > p.\ \exists u, v, w, x, y.\ P$)
- $\forall p \in \mathbb{N}^+.$ not ($\forall z \in \mathcal{L}:\ |z| > p.\ \exists u, v, w, x, y.\ P$)
- $\forall p \in \mathbb{N}^+.\ \exists z \in \mathcal{L}:\ |z| > p.$ not ($\exists u, v, w, x, y.\ P$)
- $\forall p \in \mathbb{N}^+.\ \exists z \in \mathcal{L}:\ |z| > p.\ \forall u, v, w, x, y.$ not (P)
- $\forall p \in \mathbb{N}^+.\ \exists z \in \mathcal{L}:\ |z| > p.\ \forall u, v, w, x, y.$ not ($P1$ and $P2$ and $P3$ and $P4$)
- not ($P1$ and $P2$ and $P3$ and $P4$)
- not (($P1$ and $P2$ and $P3$) and $P4$)
- not ($P1$ and $P2$ and $P3$) or not $P4$
- ($P1$ and $P2$ and $P3$) implies not $P4$
- ($P1$ and $P2$ and $P3$) implies not ($\forall i \in \mathbb{N}. uv^i wx^i y \in \mathcal{L}$)
- ($P1$ and $P2$ and $P3$) implies $\exists i \in \mathbb{N}.$ not ($uv^i wx^i y \in \mathcal{L}$)

P_1 *implies* P_2 è equivalente a $not\,(P_1) \vee P_2$.

Dunque in sintesi, la $not\,(PL\text{-}THESIS)$ è la seguente:

- $\forall p \in \mathbb{N}^+.\ \exists z \in \mathcal{L}:\ |z| > p.\ \forall u, v, w, x, y.$
  - $(z = uvwxy$ and $|vwx| \leq p$ and $|vx| > 0)$
  - implies
  - $\exists i \in \mathbb{N}. uv^i wx^i y \notin \mathcal{L}$

Dunque operativamente:

- **Qualunque** sia il numero naturale positivo p (dunque lo lasciamo "variabile")
- **Scegli** una parola z più lunga di p e appartenente al linguaggio (scegliamo quella più conveniente)
- Dimostrare che **qualunque** sia lo spacchettamento di z in $uvwxy$ con $|vxy| \leq p$ e $|vx| > 0$ che viene preso in considerazione, si può scegliere un numero naturale i che sia tale che $u v^i w x^i y \notin L$ (ovvero tale che la versione pumped della parola non appartiene al linguaggio)
  - ... let p be an arbitrary positive integer
    $\forall p$: Any p
  - Take $z = a^p b^p c^p$
    $\exists z$: Choose $z \in \mathcal{L}$ longer than p
  - ... If $(z = uvwxy$ and $|vwx| \leq p$ and $|vx| > 0)$ then ....
    $\forall u, v, w, x, y : z = uvwxy$ and $|vwx| \leq p$ and $|vx| > 0$
  - ... $uv^0wx^0y \notin \mathcal{L}$
    $\exists i$: Choose a value for the iterator
  - ... contradiction

Vediamo ora qualche esempio sul come utilizzare la $not(PL\text{-}THESIS)$.

Prendiamo in considerazione la seguente grammatica G:

$$\begin{aligned} \mathcal{G}: \quad S &\rightarrow aSBc \mid abc \\ cB &\rightarrow Bc \\ bB &\rightarrow bb \end{aligned}$$

Questa grammatica è context-dependent e il linguaggio generato è $L(G) = \{a^n b^n c^n \mid n > 0\}$.
Vogliamo verificare che non è libero tramite il pumping lemma. Supponiamo quindi che $L(G)$ sia un linguaggio libero, e sia p un intero positivo arbitrario.
Prendiamo $z = a^p b^p c^p$.
Se

- $z = uvwxy$
- $|vwx| \leq p$
- $|vx| > 0$

Allora vx non può avere occorrenze sia di a che di c perché l'ultima occorrenza di a e la prima occorrenza di c sono $p+1$ posizioni distanti.

Infatti, per alcuni k e j positivi

- O $vwx = a^k$
- O $vwx = a^k b^j$
- O $vwx = b^j$
- O $vwx = b^j c^k$
- O $vwx = c^k$

Dunque vx risulta sempre o senza occorrenze di a o senza occorrenze di c, ovvero nessuna a e nessuna c appare contemporaneamente in vx.
Quindi $u\,v^0\,w\,x^0\,y$ non può avere la forma $a^n b^n c^n$, dunque $u\,v^0\,w\,x^0\,y \notin L$.
In conclusione, per contraddizione del pumping lemma, L non è libero.

Vediamo un altro esempio con la seguente grammatica:

$$
\begin{array}{rrcl}
\mathcal{G}: & S & \rightarrow & CD \\
 & C & \rightarrow & aCA \mid bCB \mid \epsilon \\
 & AD & \rightarrow & aD \\
 & BD & \rightarrow & bD \\
 & Aa & \rightarrow & aA \\
 & Ab & \rightarrow & bA \\
 & Ba & \rightarrow & aB \\
 & Bb & \rightarrow & bB \\
 & D & \rightarrow & \epsilon
\end{array}
$$

Anche in questo caso G è context-dependent.
Analizziamo questa grammatica:

- Derived strings have a bookmark D initially at rightmost position

$$S \rightarrow CD$$

- Strings can only grow longer by replacing C

$$C \rightarrow aCA \mid bCB \mid \epsilon$$

- Non-terminals close to the rightmost delimiter can be converted to the corresponding terminal

$$
\begin{array}{rcl}
AD & \rightarrow & aD \\
BD & \rightarrow & bD
\end{array}
$$

- Non-terminals and terminals can be swapped when the terminal is at the right of the non-terminal

$$
\begin{array}{rcl}
Aa & \rightarrow & aA \\
Ab & \rightarrow & bA \\
Ba & \rightarrow & aB \\
Bb & \rightarrow & bB
\end{array}
$$

Abbiamo già visto che il linguaggio generato da questa grammatica è
$L(G) = \{ww \mid w \in (a \mid b)^*\}$.

Ci chiediamo di nuovo se questo linguaggio è libero o meno. La risposta è no e per provarlo si agisce in maniera uguale a prima (una buona scelta per z è $z = a^p b^p a^p b^p$).

Dunque:

- $\{a^n b^n c^n \mid n > 0\}$ non è libero, visto che non rispetta il pumping lemma
- $\{a^n b^n c^j \mid n, j > 0\}$ è libero in quanto è la concatenazione di due linguaggi liberi
  - $\{a^n b^n \mid n > 0\}$
  - $\{c^j \mid j > 0\}$
- $\{a^j b^n c^n \mid j, n > 0\}$ è libero in quanto è la concatenazione di due linguaggi liberi
  - $\{a^j \mid j > 0\}$
  - $\{b^n c^n \mid n > 0\}$

Questo ci conferma il fatto che **la classe dei linguaggi liberi non è chiusa rispetto all'operazione di intersezione**, infatti:

- $L_1 = \{a^n b^n c^j \mid n, j > 0\}$ è libero
- $L_2 = \{a^j b^n c^n \mid j, n > 0\}$ è libero
- $L_1 \cap L_2 = \{a^n b^n c^n \mid n > 0\}$ non è libero (per il pumping lemma)